ECOFLUX — Documento guía de diseño y continuidad

Versión 1.1 · 6 de noviembre de 2026 · Hackaton Nov 2026 Redactado por Claude. Pensado para entregarse a otra IA (GPT) o a una persona del equipo y continuar el diseño sin perder el hilo.

0. Cómo usar este documento

Si eres una IA que recibe este documento para continuar el trabajo:

1. Lee las secciones 1 a 4 completas antes de proponer nada.
2. Respeta la sección 3 (decisiones tomadas). No las reabras salvo que el usuario lo pida o encuentres un error técnico; si lo encuentras, explícalo y propón la corrección.
3. Las cifras marcadas como provisionales deben validarse con el equipo de electrónica. No las presentes como hechos.
4. Al terminar tu trabajo, actualiza la sección 16 (registro de avance) y la sección 15 (pendientes), y devuelve el documento completo.
5. Responde en español, con lenguaje claro. El usuario es estudiante de Electrónica y Automatización y trabaja en equipo.

Texto sugerido para iniciar la conversación con la otra IA:

> Te paso un documento guía de proyecto. Léelo completo y confirma en 5 líneas qué entendiste, qué decisiones están cerradas y qué está pendiente. Luego continúa desde la sección 15 (pendientes). No rehagas el análisis desde cero.

Si eres del equipo de electrónica: ve directo a la sección 7 (contrato de integración) y a la sección 18 (guía de implementación en el sistema real).

1. Contexto del reto

Hackaton de diseño y construcción de una solución tecnológica sostenible que convierta un problema real del territorio en un prototipo físico funcional, integrando hardware, software, sensores, IoT o inteligencia artificial. Metodología: Design Thinking. ODS posibles: 6 (agua limpia y saneamiento), 7 (energía asequible y no contaminante), 12 (producción y consumo responsables) y 13 (acción por el clima).

Proyecto base: informe técnico de una vivienda a escala con sistema de enfriamiento (carrera de Electrónica y Automatización, ESPAM MFL, Calceta, agosto 2026). Es un sistema 100% analógico, sin microcontrolador, hecho así por las condiciones de la

demostración original.

2. El sistema físico existente (hechos del informe)

2.1 Funcionamiento

1. Un termistor NTC de 10 kΩ dentro de la vivienda forma un divisor de tensión. Al subir la temperatura baja su resistencia y cambia el voltaje del divisor.
2. Un LM358 compara ese voltaje con una referencia ajustada por potenciómetro (10 kΩ).
3. Al cruzar el umbral, la salida del LM358 habilita etapas de potencia con MOSFET IRF640, que conmutan cargas de 12 V. Cada compuerta tiene una resistencia en serie y otra a GND.
4. Cargas: 2 ventiladores, una bomba inferior (transfiere agua del tanque inferior al elevado), una bomba sumergible de presurización (AD20P-1230C, 12 V, 3,6 W) y una electroválvula de 12 V. Los motores y la electroválvula llevan diodos de protección.
5. Circuito de agua: tanque inferior → bomba inferior → tanque elevado → bomba sumergible → electroválvula → circuito de enfriamiento.
6. Energía: 2 paneles solares de 12 V nominales y batería de 12 V y 4,5 Ah. El informe recomienda controlador de carga y fusibles, y no confirma que ya estén instalados.
7. El sensor está a unos 40 cm de la protoboard de control. No hubo interferencia apreciable.

2.2 Mediciones del informe

Medición	Valor	Observación
Panel con buena iluminación	≈ 13,49 V	medido por el estudiante
Panel dentro de la vivienda	≈ 9,57 V	medido por el estudiante
Corriente bomba inferior	≈ 0,13 A	escala de 10 A del multímetro
Corriente electroválvula	≈ 0,44 A	escala de 10 A
Bomba sumergible	12 V / 3,6 W	dato de placa
Distancia sensor–control	≈ 40 cm	sin interferencia apreciable

El propio informe pide repetir las mediciones de corriente en la escala adecuada. No hay datos de consumo de los ventiladores ni de reducción real de temperatura.

2.3 Limitaciones conocidas del sistema actual

- No hay histéresis, por lo que cerca del umbral puede haber conmutaciones rápidas.
- No hay sensor de nivel de agua.
- No hay medición de consumo ni de energía.
- El LM358 no es un comparador dedicado.
- El informe no detalla si cada bomba y la válvula tienen control independiente ni su secuencia exacta.
- No se midió cuánto baja la temperatura con el sistema encendido.

3. Decisiones tomadas

#	Decisión	Estado
D1	La capa analógica se conserva y sigue funcionando sin la ESP32 (respaldo permanente)	Cerrada
D2	Primero se diseña y construye el software de la plataforma con datos simulados; después se muestra al equipo de electrónica	Cerrada (decisión del usuario)
D3	La plataforma pide, la ESP32 decide. Toda orden pasa por reglas locales de la ESP32 y puede ser rechazada	Cerrada
D4	La IA no actúa directamente sobre actuadores. Primero explica, predice y recomienda; las acciones requieren confirmación y límites	Cerrada
D5	La interfaz es una PWA (una base de código para PC y móvil) con modo local sin internet	Propuesta, lista para desarrollar
D6	Comunicación ESP32 ↔ plataforma por MQTT con mensajes JSON	Propuesta, por validar con electrónica
D7	Un simulador de la maqueta envía el mismo formato que enviará la ESP32, de modo que cambiar uno por otro no obliga a reescribir	Propuesta, lista para desarrollar
D8	Nombre del proyecto: ECOFLUX (antes AgroClima Circular). Asistente de voz: AIRA (Asistente Inteligente de Recursos Ambientales)	Cerrada (decisión del usuario)

#	Decisión	Estado
D9	ODS principal 13 (adaptación al calor); apoyo 7, 6 y 12	Propuesta
D10	Perfil agro piloto: vivero o invernadero pequeño; galpón avícola o porcino como segundo perfil	Propuesta, por confirmar con el usuario
D11	Un perfil se construye completo; los demás se documentan como plantillas de configuración, no como funciones construidas	Propuesta
D12	Fase navegador: la plataforma se construye primero como página web (HTML y JavaScript) con el simulador dentro de la misma página y el mismo contrato de datos. La versión con Python, FastAPI y MQTT real viene después	Cerrada (decisión del usuario)
D13	Fuentes de datos intercambiables (simulador, MQTT sobre WebSocket, WebSocket directo a la ESP32, USB), con manifiesto de capacidades y validador de contrato en Diagnóstico, para pasar de simulado a real sin reescribir	Propuesta, lista para desarrollar
D14	Contrato v1.1: ts opcional y uptime_s obligatorio (la ESP32 no tiene reloj sin internet); mensaje capabilities al conectarse	Propuesta, por validar con electrónica
D15	El diseño visual se hace en Figma con tokens y componentes; el código los implementa de forma fiel	Propuesta
D16	El respaldo analógico se conserva; la orden digital entra por una compuerta OR cableada con aislamiento óptico y la seguridad de nivel corta por hardware (sección 18)	Propuesta, por validar en banco de pruebas

Lo que no se afirma sin medir: ahorro energético, reducción de temperatura y autonomía de batería. La plataforma muestra esos indicadores como "calibrando" hasta tener datos reales.

4. Definición del proyecto

ECOFLUX es una plataforma web para PC y móvil que mide, registra, explica y anticipa el comportamiento de un sistema solar de enfriamiento y gestión de agua, pensada para usuarios sin experiencia técnica.

Problema: en climas cálidos el calor interior afecta viviendas y unidades agropecuarias pequeñas, y los sistemas de enfriamiento y bombeo suelen operarse sin datos, sin alertas y sin manera sencilla de entender qué pasa.

Propuesta de valor:

- Ver en un vistazo si todo está bien (semáforo) y por qué.
- Recibir alertas en lenguaje claro antes de que algo falle.
- Consultar por voz o texto, con AIRA (Asistente Inteligente de Recursos Ambientales), el estado del sistema.
- Ver tendencias explicables de temperatura, agua y energía.
- Seguir funcionando aunque no haya internet o falle la plataforma, porque el control básico es analógico.

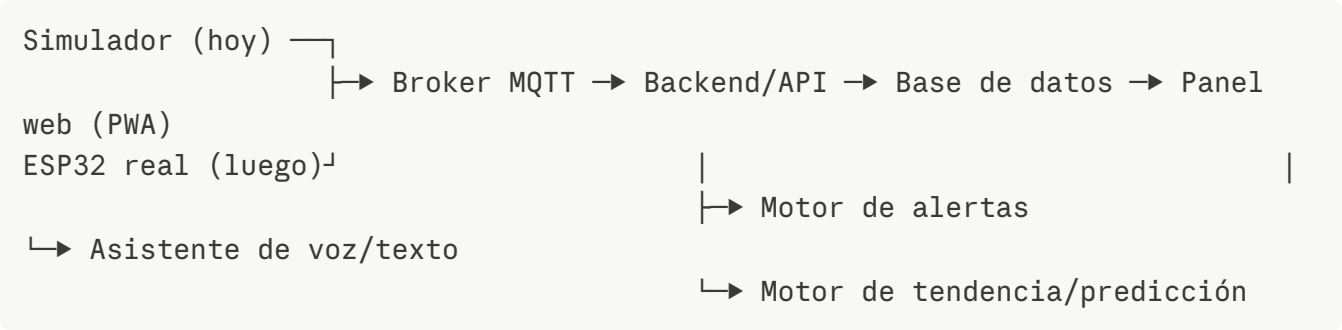
Alcance del primer entregable (MVP de software): panel, historial, alertas, tendencia y asistente de consulta, funcionando con el simulador y listo para conectar a la ESP32.

Fuera del alcance por ahora: cableado y circuito de potencia, firmware de la ESP32, control físico por IA, entrenamiento con datos reales.

5. Usuarios y tareas

Perfil	Qué necesita	Tareas principales
Productor o usuario sin experiencia	Saber si todo va bien y qué hacer	Ver semáforo, leer alertas, preguntar por voz, activar o detener el modo manual con confirmación
Administrador o técnico	Configurar y revisar	Ajustar umbrales de alerta, gestionar usuarios, exportar reportes, elegir perfil agro
Equipo de electrónica	Integrar y diagnosticar	Ver datos crudos, estado de conexión, activar el simulador, probar comandos y respuestas
Invitado o jurado	Entender el proyecto	Solo lectura, con un escenario de demostración

6. Arquitectura lógica



Pila recomendada (modificable, pero justificar el cambio):

- Broker: Mosquitto en la laptop (modo local) o un broker en la nube.
- Backend: Python con FastAPI, WebSocket hacia el panel, SQLite al inicio.
- Frontend: PWA con React y Vite, gráficas con una librería ligera, diseño móvil primero.
- Simulador: script en Python que publica por MQTT con escenarios.
- Predicción: regresión simple y series de tiempo con scikit-learn o statsmodels.
- Voz: reconocimiento y síntesis del navegador; un modelo de lenguaje con herramientas de solo lectura.

Estructura de repositorio sugerida: `/simulator`, `/backend`, `/frontend`, `/docs` (este documento y el contrato de datos).

Actualización v1.1 (fase navegador): primero se construye la plataforma como página web con HTML y JavaScript. El simulador corre dentro de la propia página y genera mensajes con el formato del contrato. La interfaz habla con una "fuente de datos" intercambiable (simulador, MQTT sobre WebSocket, WebSocket directo a la ESP32 o USB), así que pasar de simulado a real es cambiar la fuente en la pantalla de Diagnóstico, sin tocar el resto de las pantallas. La pila con Python, FastAPI y Mosquitto se mantiene como evolución posterior.

Diseño visual: las pantallas se diseñan en Figma con variables (colores de estado normal, aviso, crítico, sin dato y desconectado; tipografía; espaciado) y componentes (tarjeta de métrica, semáforo, ítem de alerta, tarjeta de gráfica, botón de voz de AIRA, franja de modo). Cada componente tiene sus estados y tres tamaños: móvil (390 px), tablet y escritorio (1440 px). Una franja permanente indica "MODO SIMULADO" o "CONECTADO A ESP32 REAL".

7. Contrato de integración con la ESP32

Todo es una propuesta para revisar con electrónica. Los nombres, unidades y frecuencias pueden cambiar, pero cualquier cambio debe actualizarse aquí.

7.1 Temas MQTT

Identificador de unidad: `{unit_id}`, por ejemplo `maqueta01`.

Tema	Sentido	Contenido
`ecoflux/{unit_id}/telemetry`	ESP32 → plataforma	Mediciones y estados
`ecoflux/{unit_id}/events`	ESP32 → plataforma	Eventos puntuales
`ecoflux/{unit_id}/status`	ESP32 → plataforma	En línea o fuera de línea (mensaje de última voluntad)
`ecoflux/{unit_id}/cmd`	plataforma → ESP32	Comandos
`ecoflux/{unit_id}/cmd/ack`	ESP32 → plataforma	Aceptado o rechazado, con motivo

Todo mensaje incluye `schema_version`, `unit_id` y `uptime_s` (segundos desde el arranque). El campo `ts` (hora UTC en formato ISO 8601) es opcional: la ESP32 solo lo envía si tiene la hora sincronizada por NTP; si falta, la plataforma registra la hora de recepción.

Desde la versión 1.1 el prefijo de los temas es `ecoflux/` (antes `agroclima/`).

7.2 Telemetría

Frecuencia base: cada 5 s. Los estados de actuadores se envían además al cambiar.

Campo	Unidad	Origen	Notas
`temp_int_c`	°C	NTC acondicionado o sensor digital	Requiere calibración con termómetro de referencia
`hum_int_pct`	%	Sensor de humedad (opcional, p. ej. DHT22)	Puede faltar en la maqueta
`tank_low_pct`	%	Sensor de nivel, tanque inferior	Aún no existe en la maqueta
`tank_high_pct`	%	Sensor de nivel, tanque elevado	Aún no existe en la maqueta
`batt_v`	V	Medición con divisor protegido	
`batt_a`	A	Sensor de corriente (signo: positivo carga)	
`panel_v`	V	Medición con divisor protegido	
`panel_a`	A	Sensor de corriente	
`comparator_on`	bool	Observación de la salida del LM358 (acondicionada)	Estado del control analógico
`mode`	texto	ESP32	`auto` o `manual`
`loads`	objeto	ESP32	Un objeto por carga, ver abajo
`rssi_dbm`	dBm	ESP32	Calidad de Wi-Fi
`fw_version`	texto	ESP32	
uptime_s	s	ESP32	Segundos desde el arranque; permite ordenar mensajes cuando la ESP32 no tiene hora

Cada carga (`fan1` , `fan2` , `pump_low` , `pump_high` , `valve`):

Subcampo	Descripción
`on`	Estado observado (bool)
`current_a`	Corriente medida, si hay sensor (A)
`cmd_source`	Quién la mandó: `analog`, `digital`, `manual` o `unknown`

Cualquier campo que no se pueda medir se envía como `null`, nunca se inventa. La plataforma muestra "sin dato".

7.3 Eventos

`event`	Cuándo
`boot`	Arranque o reinicio de la ESP32
`sensor_lost`	Un sensor deja de responder (campo en `detail`)
`emergency_stop`	Parada de emergencia física o por comando
`cmd_rejected`	Se rechazó un comando (motivo en `detail`)
`mode_changed`	Cambio de modo
`safety_cutoff`	Corte por seguridad local (nivel bajo, tiempo máximo, sobrecorriente)

7.4 Comandos y respuesta

`cmd`	Parámetros	Condiciones para aceptar (todas provisionales)
`set_mode`	`mode`: `auto` o `manual`	Siempre aceptado, se registra
`request_fan`	`fan_id`, `duration_s`	Solo en `manual`; duración máxima 600 s; rechazar si falla el sensor de temperatura
`request_pump`	`pump_id`, `duration_s`	Fase posterior. Solo en `manual`; nivel del tanque de origen sobre el mínimo; duración máxima 300 s

cmd	Parámetros	Condiciones para aceptar (todas provisionales)
`suggest_threshold`	`value_c`	Solo se registra como sugerencia; el umbral real lo sigue fijando el potenciómetro hasta que electrónica decida otra cosa
`emergency_stop`	ninguno	Siempre aceptado; apaga todas las cargas gobernadas digitalmente

Respuesta en `cmd/ack`: `cmd_id`, `accepted` (bool) y `reason`. Códigos de rechazo: `MODE_NOT_MANUAL`, `LIMIT_DURATION`, `LEVEL_LOW`, `SENSOR_FAULT`, `UNKNOWN_CMD`, `SAFETY_LOCK`.

7.5 Arbitraje entre mando analógico y digital

Es una propuesta por validar en banco de pruebas, no algo verificado.

Situación	Regla propuesta
Temperatura sobre el umbral	Manda el analógico. La ESP32 no apaga lo que el analógico encendió por temperatura
Preenfriamiento predicho	La ESP32 solo puede pedir ventiladores, con tiempo máximo
Seguridad de bombas	Cualquiera de los dos lados puede cortar. El flotador de nivel debe actuar por cableado, sin depender de la ESP32
Falla o reinicio de la ESP32	Las salidas caen a estado seguro y el analógico conserva el mando
Orden manual desde app o voz	Con confirmación y límite de tiempo, empezando solo por ventiladores

Advertencia para electrónica: no basta con unir señales o compuertas. Hace falta un circuito de coordinación e interbloqueo diseñado y probado.

7.6 Puntos de integración que electrónica debe resolver

- Medir el voltaje real del nodo del NTC y acondicionarlo antes de la entrada ADC de la ESP32. No conectar el NTC directo.
- Usar pines ADC1 si se usa Wi-Fi (el ADC2 es poco fiable con Wi-Fi activo).

- Revisar la compuerta del IRF640: con 3,3 V no conduce bien; usar relé, optoacoplador, MOSFET de nivel lógico o un driver.
- Medir la corriente de cada carga, incluidos ventiladores, y su corriente de arranque.
- Confirmar la química de la batería y si hay controlador de carga, para fijar los umbrales de batería.
- Compartir GND entre etapas y separar la masa de potencia de la de señal.

7.7 Manifiesto de capacidades

Al conectarse y después de cada reinicio, la ESP32 publica un mensaje retenido en `ecoflux/{unit_id}/capabilities`. Dice qué existe realmente en esa unidad. La plataforma muestra solo esos campos y controles, y marca el resto como "sin dato". Así electrónica puede añadir sensores poco a poco sin cambiar la plataforma.

```
{
  "schema_version": "1.1",
  "unit_id": "maqueta01",
  "uptime_s": 12,
  "fw_version": "0.1.0",
  "fields": ["temp_int_c", "hum_int_pct", "batt_v", "batt_a", "panel_v",
"panel_a", "comparator_on"],
  "loads": ["fan1", "fan2", "pump_low", "pump_high", "valve"],
  "loads_controllable": [],
  "limits": {"fan_max_s": 600, "pump_max_s": 300}
}
```

- `fields`: campos de telemetría que esta unidad sí mide. Los que no estén aquí se muestran como "sin dato".
- `loads_controllable`: cargas que la ESP32 acepta pedir de forma digital. Vacía en la fase de observación.
- `limits`: valores que la ESP32 aplica de verdad. La plataforma los muestra, no los impone.

7.8 Fuentes de datos y validador de contrato

La plataforma habla con una sola "fuente de datos" y puede cambiarla desde la pantalla de Diagnóstico. Todas usan los mismos mensajes JSON.

Fuente	Cómo conecta	Qué debe hacer la ESP32
Simulador	Código dentro de la propia página	Nada. Sirve para desarrollar y demostrar sin hardware

Fuente	Cómo conecta	Qué debe hacer la ESP32
MQTT sobre WebSocket	La ESP32 publica por MQTT normal al broker; el navegador se conecta al mismo broker por WebSocket	Publicar `telemetry`, `events`, `status` (con mensaje de última voluntad) y `capabilities`; suscribirse a `cmd`
WebSocket directo	La ESP32 sirve el panel y un WebSocket propio, sin router ni internet	Enviar y recibir los mismos JSON por el socket. Pocos clientes a la vez
USB (Web Serial)	Cable entre la ESP32 y un PC con navegador Chromium	Una línea JSON por mensaje a 115200 baudios, con un campo `topic` que indica el tipo (`telemetry`, `events`, `ack`). Los comandos llegan por la misma vía

El validador de contrato, dentro de Diagnóstico, revisa cada mensaje recibido y marca en verde o rojo:

- JSON válido, `schema_version` y `unit_id` presentes.
- Periodo de telemetría cercano a 5 s (tolerancia del 20 %).
- Rangos plausibles (por ejemplo, temperatura entre -10 y 80 °C, batería entre 0 y 20 V, porcentajes entre 0 y 100). Provisional.
- Campos no medidos enviados como `null`, nunca inventados.
- Cada comando recibe un `ack` en menos de 2 s.
- `status` pasa a fuera de línea cuando la ESP32 se desconecta.

Para que la conexión funcione, hay que decidir el despliegue desde ahora. Una página alojada por `https://` solo puede abrir `wss://` (broker con TLS). Una página servida por `http://` en la red local puede usar `ws://`. La ESP32 solo usa Wi-Fi de 2,4 GHz; las redes de campus con aislamiento de clientes o portal cautivo suelen fallar, y para la demo conviene un hotspot del celular o un router propio.

7.9 Ejemplos de mensajes

Los valores son de ejemplo, no son mediciones del sistema.

Telemetría (`ecoflux/maqueta01/telemetry`):

```
{
  "schema_version": "1.1",
  "unit_id": "maqueta01",
  "uptime_s": 3605,
  "temp_int_c": 31.4,
```

```
  "hum_int_pct": 62.0,
  "tank_low_pct": null,
  "tank_high_pct": null,
  "batt_v": 12.4,
  "batt_a": -0.9,
  "panel_v": 13.5,
  "panel_a": 0.4,
  "comparator_on": true,
  "mode": "auto",
  "loads": {
    "fan1": {"on": true, "current_a": 0.15, "cmd_source": "analog"},
    "fan2": {"on": true, "current_a": 0.15, "cmd_source": "analog"},
    "pump_low": {"on": false, "current_a": 0.0, "cmd_source": "unknown"},
    "pump_high": {"on": false, "current_a": null, "cmd_source": "unknown"},
    "valve": {"on": false, "current_a": null, "cmd_source": "unknown"}
  },
  "rssi_dbm": -61,
  "fw_version": "0.1.0"
}
```

Evento (`ecoflux/maqueta01/events`):

```
{"schema_version": "1.1", "unit_id": "maqueta01", "uptime_s": 3610, "event":
"cmd_rejected", "detail": "LEVEL_LOW"}
```

Comando (`ecoflux/maqueta01/cmd`) y su respuesta (`ecoflux/maqueta01/cmd/ack`):

```
{"schema_version": "1.1", "unit_id": "maqueta01", "cmd_id": "c-0007", "cmd":
"request_fan", "fan_id": "fan1", "duration_s": 300}
{"schema_version": "1.1", "unit_id": "maqueta01", "uptime_s": 3611, "cmd_id":
"c-0007", "accepted": false, "reason": "MODE_NOT_MANUAL"}
```

Estado (`ecoflux/maqueta01/status` , retenido): el texto `online` al conectarse y `offline` como mensaje de última voluntad.

8. Pantallas y funciones

#	Pantalla	Contenido
S1	Inicio	Semáforo general, temperatura y humedad, estado de ventiladores, bombas y válvula, nivel de agua, batería, últimas alertas, botón de modo y de parada

#	Pantalla	Contenido
S2	Historial	Gráficas por día y semana de temperatura, consumo, agua y energía; comparación entre periodos; exportar CSV
S3	Alertas	Lista con severidad, explicación en lenguaje simple y qué hacer
S4	Tendencia	Pronóstico a 30 y 60 min con la gráfica, los datos usados y un indicador de confianza
S5	Asistente	Voz o texto: consultar estado, explicar alertas y eventos
S6	Ajustes y perfiles	Umbrales de alerta, perfil agro, usuarios y roles, idioma
S7	Diagnóstico	Datos crudos, estado de conexión, interruptor del simulador, prueba de comandos y respuestas

Principios de diseño para usuarios sin experiencia: color acompañado de texto e icono, frases cortas, un solo botón principal por pantalla, confirmación antes de cualquier acción física, y todo legible en móvil.

9. Reglas de alertas (provisionales)

Los umbrales numéricos se calibran con datos reales y con electrónica.

ID	Condición	Severidad	Mensaje sugerido
A01	`temp_int_c` sobre el umbral de alerta durante más de 10 min	Aviso, luego Crítica a los 30 min	"La temperatura interior sigue alta. Revisa los ventiladores."
A02	`tank_high_pct` bajo 20 %	Aviso	"El tanque elevado se está quedando sin agua."
A03	Una bomba encendida con corriente muy por debajo de su valor típico más de 10 s	Crítica	"Una bomba parece funcionar sin agua. Apágala."
A04	`batt_v` bajo 11,5 V (Aviso) o bajo 11,0 V (Crítica); valores para batería de plomo, a confirmar	Aviso o Crítica	"La batería está baja."

ID	Condición	Severidad	Mensaje sugerido
A05	Sin telemetría durante 3 periodos	Aviso	"Se perdió la conexión con el sistema."
A06	Más de 6 conmutaciones de una carga en 10 min	Aviso	"El sistema se enciende y apaga muy seguido cerca del umbral."
A07	`panel_v` bajo en horas de sol	Informativa	"Los paneles generan poco. Revisa sombras o suciedad."

Las alertas A03 y A06 sirven además para evidenciar a electrónica la necesidad de sensor de nivel e histéresis.

10. Tendencia y predicción

Principio: que sea explicable. Cada estimación muestra la gráfica, los datos usados y la confianza.

- v0: promedio móvil exponencial y pendiente de los últimos 30 min para estimar la temperatura a +30 y +60 min. Se presenta como "tendencia", no como predicción.
- v1: regresión con variables como hora del día, voltaje del panel como indicador de radiación y temperatura de las últimas horas. Se evalúa con error medio absoluto frente a la regla simple v0; solo reemplaza a v0 si la supera.
- Detección de anomalías: comparar corriente típica de cada bomba contra la observada (ligado a A03).
- Autonomía de batería: no se muestra al usuario hasta calibrarla con mediciones reales de corriente acumulada. Mientras tanto aparece "calibrando".
- Preenfriamiento: la plataforma solo sugiere (comando `request_fan`) y la ESP32 decide.

11. AIRA, el asistente de voz con IA

AIRA (Asistente Inteligente de Recursos Ambientales) es el nombre del asistente de ECOFLUX. Se activa con un botón o diciendo su nombre cuando el navegador lo permita, y se presenta con una frase corta como "Soy AIRA, tu asistente de ECOFLUX". Habla en español claro y sigue las reglas de esta sección: en la fase 1 solo consulta, y en la fase 2 propone acciones que el usuario confirma.

Fase 1 (solo consulta): intenciones como "¿cómo está la temperatura?", "¿por qué se encendió la bomba?", "¿cuánta agua queda?", "¿qué alertas hay?", "¿cómo va la tendencia?". El modelo de lenguaje usa funciones de solo lectura: `get_status()`, `get_history(rango)`, `get_alerts()`, `explain_event(id)`, `get_forecast()`.

Fase 2 (acciones con confirmación): el asistente propone una acción, el panel muestra una tarjeta de confirmación y el usuario acepta o cancela. El comando sigue el flujo de la sección 7.4 y puede ser rechazado. El asistente informa el resultado, aceptado o rechazado, con el motivo en lenguaje simple.

Reglas: responde en español claro; si no tiene el dato, dice "sin dato" y no estima; no ejecuta nada sin confirmación; si no hay internet, ofrece solo texto con respuestas locales básicas.

12. Modo sin internet

- La PWA guarda en caché la interfaz y los últimos datos.
- El broker y el backend pueden correr en una laptop o un equipo local dentro de la misma red.
- La ESP32 mantiene su control local y guarda datos temporalmente si pierde la conexión, y los sincroniza al volver.
- Más adelante: panel mínimo servido por la propia ESP32 en modo punto de acceso.

13. Simulador de la maqueta

Objetivo: poder desarrollar y demostrar sin hardware. Publica los mismos temas y campos de la sección 7.

Versión 1.1: el simulador corre primero dentro de la página web, en JavaScript, y publica el mismo contrato de datos de la sección 7. La versión en Python que publica por MQTT queda para la etapa de conexión real.

Modelo simple:

- Temperatura exterior sinusoidal diaria con ruido.
- Temperatura interior como sistema de primer orden que sigue a la exterior, con enfriamiento cuando hay ventiladores o agua circulando.
- Control analógico simulado: enciende cargas al superar el umbral, con y sin histéresis (para mostrar el problema).
- Batería que se carga según `panel_v` y se descarga según las cargas.
- Niveles de agua que cambian cuando las bombas funcionan.

Escenarios para demo y pruebas:

Escenario	Qué demuestra
Día normal	Operación estándar y tendencia
Ola de calor	Alertas A01 y preenfriamiento sugerido
Tanque que se vacía	Alerta A02 y rechazo `LEVEL_LOW`
Bomba sin agua	Alerta A03
Falla de sensor	`sensor_lost` y "sin dato"
Pérdida de conexión	Alerta A05 y modo sin internet
Conmutación rápida	Alerta A06

14. Hoja de ruta de desarrollo (software)

Etapa	Entregable	Criterio de aceptación
S0	Contrato de datos validado en este documento y repositorio base	Electrónica lo revisa y marca los campos viables
S1	Simulador con 2 escenarios y broker local	Se ven mensajes con el formato del contrato
S2	Backend con almacenamiento y WebSocket; panel S1 (Inicio) y S7 (Diagnóstico)	La demo muestra datos en vivo en PC y móvil
S3	S2 (Historial) y S3 (Alertas) con reglas A01 a A07	Cada escenario dispara su alerta
S4	S4 (Tendencia v0) con indicador de confianza	La gráfica muestra datos usados y error medido
S5	S5 (Asistente) en modo consulta	Responde las intenciones de la sección 11 sin inventar datos
S6	S6 (Ajustes y perfiles) y primer perfil agro completo	El perfil cambia umbrales, alertas y textos
S7	Paquete para electrónica: contrato, simulador y guía de conexión	Un técnico conecta la ESP32 sin leer el código del backend

Si el tiempo es corto, llegar a S4 ya es una demo sólida. Con S1 y S2 hay algo para mostrar al equipo de electrónica.

Fase navegador (decidida en la versión 1.1): se avanza por versiones de la página web, que equivalen a las etapas anteriores.

Versión navegador	Equivale a	Contenido
B1	S1 y S2	Simulador, pantalla de Inicio, Diagnóstico y escenarios
B2	S3	Historial y alertas A01 a A07
B3	S4	Tendencia con indicador de confianza
B4	S5	AIRA en modo consulta, por texto y por voz
B5	S6	Ajustes y primer perfil agro

La conexión real (broker MQTT y paquete para electrónica, S7) se hace después, cambiando la fuente de datos.

15. Pendientes y preguntas abiertas

Para el usuario o el equipo:

1. Confirmar el perfil agro piloto (D10) y si hay un vivero o invernadero accesible para pruebas.
2. Fecha de entrega del hackaton y tamaño del equipo de software, para ajustar la hoja de ruta.
3. Elegir lenguaje y pila si el equipo prefiere otra (por ejemplo, otra opción en lugar de React).
4. Confirmar si la demo se presentará con simulador, con maqueta o ambos.

Para electrónica (cuando se les muestre la plataforma):

1. Qué campos de la telemetría son viables hoy y cuáles necesitan sensores nuevos.
2. Voltaje real del nodo del NTC y cómo acondicionarlo.
3. Corriente de cada carga, incluidos ventiladores y corrientes de arranque.
4. Química de la batería y existencia de controlador de carga.
5. Diseño y prueba del interbloqueo entre mando analógico y digital.
6. Valores definitivos de límites de tiempo y umbrales de alerta.

Técnicos abiertos del diseño de software:

1. Autenticación y roles (método y almacenamiento de usuarios).
2. Política de retención del historial y exportación de reportes.

3. Elección del modelo de lenguaje y control de costos de la función de voz.
4. Cómo servir el panel mínimo desde la propia ESP32.

Nuevos pendientes (v1.1):

- Compartir el enlace del archivo de Figma con el diseño de la pantalla de Inicio, o confirmar que se parte de cero.
- Decidir dónde se aloja la página para la conexión real (laptop en la red local o GitHub Pages) y qué broker se usa, porque eso define si hace falta TLS (`wss://`).
- Confirmar la red Wi-Fi de la demo (hotspot del celular o router propio).
- Con electrónica: revisar la sección 18 y marcar qué módulos y fases pueden implementar y cuándo.
- Definir cómo se activa AIRA por voz (botón, nombre o ambos) y qué servicio de lenguaje usará, cuidando que la clave nunca quede en el código del navegador.

16. Registro de avance

Fecha	Quién	Qué se hizo
2026-11-06	Claude	Análisis del informe, comparación con el análisis de ChatGPT, definición del proyecto y creación de este documento (versión 1.0)
2026-11-06	Claude	Versión 1.1: nombre ECOFLUX y asistente AIRA; fase navegador; fuentes de datos intercambiables; contrato v1.1 (ts opcional, uptime_s, capabilities); guía de implementación para electrónica (sección 18). Secciones modificadas: 0, 3, 4, 6, 7, 11, 13, 14, 15, 16, 17 y nueva 18

Al continuar el trabajo, añadir una fila por sesión con fecha, quién la hizo y qué cambió, e indicar qué secciones se modificaron.

17. Glosario

- ADC: convertidor analógico a digital de la ESP32.
- Histéresis: diferencia entre el umbral de encendido y el de apagado, para evitar conmutaciones rápidas.
- Interbloqueo: circuito o lógica que impide órdenes contradictorias sobre una misma carga.
- MQTT: protocolo ligero de mensajes entre dispositivos.

- PWA: aplicación web instalable que funciona en PC y móvil, con caché para uso sin conexión.
- Telemetría: mediciones que el dispositivo envía de forma periódica.
- AIRA: Asistente Inteligente de Recursos Ambientales, el asistente de voz de ECOFLUX.
- ECOFLUX: nombre del proyecto.
- Fuente de datos: pieza que entrega los mensajes del contrato a la plataforma (simulador, MQTT sobre WebSocket, WebSocket directo o USB).
- Manifiesto de capacidades: mensaje `capabilities` donde la ESP32 declara qué mide y qué puede controlar.
- Mensaje de última voluntad (LWT): mensaje MQTT que el broker publica cuando la ESP32 se desconecta sin avisar.
- Compuerta OR cableada: unión de dos mandos con diodos, de modo que cualquiera de los dos pueda encender una carga.
- Optoacoplador: componente que transmite una señal con luz, para aislar eléctricamente la ESP32 del circuito de 12 V.
- Web Serial: función de los navegadores Chromium para leer un puerto USB desde la página.

18. Guía de implementación para el equipo de electrónica

Esta sección explica cómo llevar el contrato de la sección 7 al sistema real sin perder el respaldo analógico. Todo es una propuesta para revisar y validar en banco de pruebas. Los valores de componentes son puntos de partida y no cálculos definitivos, porque el informe no incluye el esquema eléctrico completo.

18.1 Principios de integración

1. El sistema analógico no se modifica en la fase de observación. La ESP32 solo mide.
2. Ningún pin de la ESP32 recibe más de 3,3 V. Todo lo que viene del lado de 12 V pasa por un módulo de medición, un divisor protegido o un aislamiento óptico.
3. La ESP32 solo puede pedir. Su orden se suma al mando analógico (compuerta OR) y nunca lo apaga.
4. La seguridad del agua (nivel bajo) corta por hardware, sin depender de la ESP32.
5. Si la ESP32 se reinicia o pierde la conexión, las salidas digitales quedan apagadas y el analógico sigue mandando.
6. Cada fase se prueba con las pruebas de la sección 18.8 antes de pasar a la siguiente.

18.2 Fases de implementación

Fase	Qué se implementa	Se considera lista cuando
E0. Mediciones base	Corriente de cada carga (reposo y arranque) en la escala adecuada; voltaje del nodo del NTC y de la salida del LM358, frío y caliente; temperatura de activación y desactivación con termómetro de referencia	La tabla de mediciones de la sección 18.7 está completa y archivada
E1. Observación	ESP32 con sensor digital de temperatura, voltaje y corriente de panel y batería, lectura aislada de la salida del LM358, y publicación de `telemetry`, `status` y `capabilities`	El validador de contrato está en verde y las lecturas coinciden con el multímetro
E2. Agua y corrientes por carga	Niveles de tanque, corriente por carga y corte por hardware de bombas con nivel bajo	Con el tanque bajo, la bomba no arranca aunque el analógico lo pida
E3. Mando digital de ventiladores	Salidas aisladas en compuerta OR para `fan1` y `fan2`; comandos `set_mode`, `request_fan` y `emergency_stop`; límites de tiempo	Los comandos se aceptan o se rechazan con el `reason` correcto y el analógico sigue mandando
E4. Mejoras del lado analógico	Histéresis, dos umbrales, controlador de carga solar y fusibles (sección 18.5 y 18.6)	Sin conmutaciones rápidas durante 30 min cerca del umbral
E5. Bombas y electroválvula digitales (opcional)	El mismo esquema de compuerta OR para bombas y electroválvula	Pruebas de la sección 18.8 superadas con agua

18.3 Lista de componentes sugerida

Función	Componente sugerido	Notas
Microcontrolador	ESP32 DevKit (ESP32-WROOM-32)	Revisar la distribución de pines de la placa concreta
Alimentación de la ESP32	Convertidor reductor (buck) de 12 V a 5 V, de 1 A o más	Evitar reguladores lineales, que disipan mucho al bajar de 12 V. Condensadores de 470 µF y 100 nF junto a la ESP32
Voltaje y corriente	Módulos INA219 (I2C): uno para el panel, uno para la batería y, si se quiere, uno por rama de carga	Bus hasta 26 V y, con la resistencia de 0,1 Ω, hasta unos 3,2 A. Direcciones I2C 0x40 a 0x43 con los puentes A0 y A1
Temperatura y humedad interior	DHT22 (o SHT31 por I2C)	Sensor independiente del NTC, para no cargar el nodo analógico. El DHT22 se lee como máximo cada 2 s
Lectura de la salida del LM358	Optoacoplador PC817 con resistencia en serie	Protege la ESP32 de los 12 V. Se publica como `comparator_on`
Orden digital hacia cada carga	Optoacoplador (PC817 o similar) y diodo 1N4148 hacia la compuerta del MOSFET	Compuerta OR cableada (sección 18.4)
Nivel de agua	Interruptores de flotador (mínimo y máximo)	En la maqueta bastan niveles discretos; si se envían como porcentaje, mapear a 0, 50 y 100 %. Alternativa: sensor capacitivo sin contacto
Corte de seguridad por nivel	Transistor pequeño (2N7000 o 2N2222) que lleva la compuerta del MOSFET a GND	Lo activa el circuito del flotador, sin depender de la ESP32
Parada de emergencia	Pulsador físico normalmente cerrado en serie con el +12 V de los optoacopladores de salida	Corta toda orden digital. Un segundo contacto informa a la ESP32
Caudal (opcional)	YF-S201	Comprobar que el caudal de la bomba de la maqueta entra en su rango; si no, usar la firma de corriente de la bomba

Función	Componente sugerido	Notas
Protecciones	Fusibles por rama, diodo Schottky contra polaridad inversa, controlador de carga solar	Dimensionar con las corrientes medidas en E0

18.4 Conexiones propuestas

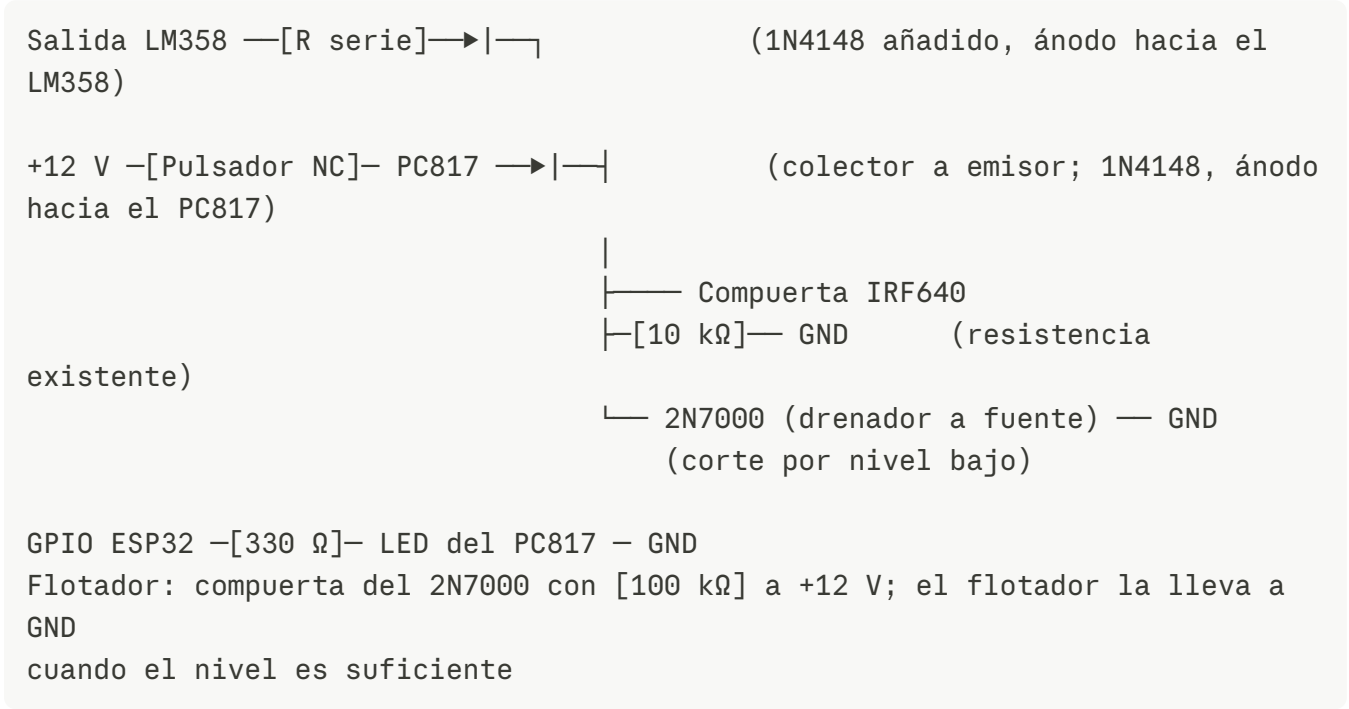
Mapa de pines propuesto. Es provisional y debe revisarse contra la placa concreta.

Señal	Pin ESP32	Tipo	Nota
I2C SDA y SCL	GPIO 21 y GPIO 22	I2C	INA219 y, si se usa, SHT31. Resistencias de 4,7 kΩ a 3,3 V si los módulos no las traen
DHT22	GPIO 4	Entrada digital	Resistencia de 10 kΩ a 3,3 V
Salida del LM358 (por optoacoplador)	GPIO 13	Entrada con pull-up interno	Activo en bajo: 0 significa comparador activado
Flotador, tanque inferior	GPIO 32	Entrada con pull-up interno	Contacto independiente del que usa el corte por hardware
Flotador, tanque elevado	GPIO 33	Entrada con pull-up interno	Igual que el anterior
Pulsador de emergencia (segundo contacto)	GPIO 23	Entrada con pull-up interno	Informa el evento `emergency_stop`
Orden `fan1`	GPIO 25	Salida al LED del optoacoplador	Fase E3
Orden `fan2`	GPIO 26	Salida al LED del optoacoplador	Fase E3
Orden `pump_low`	GPIO 27	Salida al LED del optoacoplador	Fase E5
Orden `pump_high`	GPIO 18	Salida al LED del optoacoplador	Fase E5
Orden `valve`	GPIO 19	Salida al LED del optoacoplador	Fase E5

Pines a evitar: GPIO 0, 2, 12 y 15 (definen el arranque), GPIO 6 a 11 (memoria flash) y GPIO 34 a 39 como salidas (solo son entradas). Con Wi-Fi activo, las lecturas analógicas deben usar ADC1.

Lectura de la salida del LM358 (fase E1). La salida del LM358 va por una resistencia de 1 kΩ al LED de un PC817, con el cátodo a GND. El colector va al GPIO 13. Con la salida alta de unos 10 V, la corriente del LED es de unos 9 mA; el valor real de la salida se mide en E0.

Orden digital en compuerta OR (fases E3 y E5). Cada orden digital entra al nodo de compuerta del MOSFET a través de un optoacoplador y un diodo. Se añade un diodo 1N4148 en el camino del LM358 para que, cuando su salida esté en bajo, no arrastre la compuerta hacia GND. La compuerta pierde unos 0,6 V del nivel del LM358, lo que no afecta a cargas de esta corriente; se confirma con una medición.



- El LED del optoacoplador solo conduce si el GPIO está en alto. Con la ESP32 reiniciándose o desconectada, la orden digital queda apagada. Se añade una resistencia de 10 kΩ de cada GPIO de salida a GND para que no flote durante el arranque.
- El corte por nivel es a prueba de fallos: si el flotador marca nivel bajo o el cable se rompe, la compuerta del 2N7000 queda en alto y bloquea la compuerta del MOSFET de la bomba. Esto vale para el analógico y para el digital.
- El pulsador de emergencia normalmente cerrado corta el +12 V que alimenta los optoacopladores de salida. Ninguna orden digital pasa mientras esté presionado.
- No conectar al GPIO el nodo de 12 V del flotador. La ESP32 lee el nivel con un segundo contacto o un sensor independiente a 3,3 V.

- Origen de la orden (`cmd_source`): `analog` si el comparador está activo (el analógico tiene prioridad), `digital` si solo hay orden de la ESP32, `manual` si la orden vino de un comando en modo manual y `unknown` si no hay forma de saberlo.

18.5 Mejoras en el lado analógico

Son las mejoras que el propio informe deja pendientes. Se implementan en la fase E4, después de la observación.

Histéresis en el LM358. Se añade una resistencia de realimentación positiva (Rf) desde la salida del LM358 a la entrada no inversora. Referida al voltaje de la señal, la histéresis es aproximadamente ΔV = ΔVsalida × Rth / Rf, con Rf mucho mayor que Rth. Rth es la resistencia equivalente que ve esa entrada, y ΔVsalida es la diferencia entre la salida alta y la baja (unos 9 a 10 V con 12 V de alimentación; se mide en E0).

Ejemplo con supuestos que hay que comprobar: NTC de 10 kΩ (β cercano a 3950) con una resistencia fija de 10 kΩ desde 12 V. Cerca de 30 °C la señal cambia unos 0,13 V por °C. Para una histéresis de 1,5 °C (unos 0,2 V) con Rth de unos 4,5 kΩ, Rf resulta de unos 220 kΩ. Se recomienda montar un preset de 470 kΩ, ajustar mirando las temperaturas de encendido y apagado, y después fijar el valor.

Dos umbrales. Usar el segundo amplificador del LM358 (el informe no aclara si está libre) con otro potenciómetro: un umbral más bajo para ventiladores y uno más alto para activar el circuito hidráulico.

Comparador dedicado. Si se rediseña, un LM393 (salida a colector abierto con resistencia pull-up) está pensado para comparar. El LM358 funciona, pero no es un comparador dedicado.

El NTC y la ESP32. La ESP32 no se conecta al nodo del NTC. La temperatura digital sale de un sensor independiente (DHT22 o SHT31) colocado junto al NTC. Si se quiere comparar ambas lecturas, el nodo del NTC se lee con un seguidor de tensión y un divisor hacia un pin ADC1, sin cargar el nodo ni superar 3,3 V.

18.6 Alimentación y protección

- Tierra en estrella: la tierra de potencia (motores, bombas, válvula) vuelve al negativo de la batería por su propio camino. La tierra de señal (ESP32, sensores, LM358) se une a la de potencia en un solo punto.
- La ESP32 se alimenta desde el buck, con su propia rama con fusible (por ejemplo 0,5 A, a confirmar en E0) y con los condensadores de 470 µF y 100 nF. No alimentarla a la vez por USB y por el buck sin un diodo que evite devolver los 5 V.
- Un fusible general cerca del positivo de la batería y uno por rama de carga, de 1,5 a 2 veces la corriente medida en E0, incluida la de arranque.

- Controlador de carga solar adecuado para la química de la batería (confirmar si es de plomo y revisar su hoja de datos). Para una batería de plomo de 4,5 Ah, una referencia habitual es limitar la corriente de carga a unos 0,3 C, es decir, unos 1,35 A. No conectar el panel directamente a la batería.
- Corte por bajo voltaje de la batería por hardware. En baterías de plomo suele estar entre 10,5 y 11 V; confirmar con la hoja de datos. La ESP32 solo vigila y alerta (A04).
- Diodo Schottky o MOSFET en la entrada de 12 V contra polaridad inversa.
- La electrónica va en una caja cerrada y alta, lejos de tanques y tuberías, con los cables de agua formando goteo hacia abajo.

18.7 Mediciones de la fase E0

Medición	Cómo	Dónde se usa
Corriente de cada carga, en reposo y en el arranque	Multímetro en serie en la escala adecuada o INA219; anotar el valor estable y el pico	Campo `current_a`, alerta A03 y fusibles
Voltaje del nodo del NTC	Multímetro entre el nodo y GND a 25, 30, 35 y 40 °C con termómetro de referencia	Calibración de `temp_int_c` y cálculo de histéresis
Voltaje de la salida del LM358, alto y bajo	Multímetro con el comparador activado y desactivado	Resistencia del optoacoplador y diseño de histéresis
Temperatura de activación y desactivación	Al menos 5 repeticiones con termómetro de referencia	Umbral real y histéresis necesaria
Voltaje del panel y corriente de carga	Multímetro o INA219, a pleno sol, con sombra y en interior	Alerta A07 y modelo de predicción
Batería en reposo y bajo carga	Multímetro; anotar la química y la hoja de datos	Alerta A04 y autonomía
Consumo total durante una ronda completa	INA219 en la batería	Autonomía real
Temperatura con el sistema apagado y encendido	Registro cada minuto durante cada prueba	Comparación antes y después

18.8 Pruebas de aceptación

Los criterios numéricos son provisionales y los fija electrónica.

- [ ] P1. Temperatura, voltajes y corrientes coinciden con un instrumento de referencia dentro de la tolerancia acordada.
- [ ] P2. El validador de contrato se mantiene en verde durante 30 min seguidos.
- [ ] P3. Al perder el Wi-Fi, el analógico sigue funcionando; al volver, la ESP32 se reconecta y la plataforma muestra los datos.
- [ ] P4. Reiniciar la ESP32 con cargas encendidas por el analógico no apaga ni enciende nada por error.
- [ ] P5. Con el tanque bajo, la bomba no arranca ni por el analógico ni por el digital.
- [ ] P6. Los comandos se rechazan con el `reason` correcto: fuera de modo manual, duración excedida y sensor de temperatura en fallo.
- [ ] P7. La parada de emergencia corta las órdenes digitales y la ESP32 informa el evento.
- [ ] P8. Con ventiladores y bombas encendidos, la lectura de temperatura no salta más de 0,5 °C sin causa.
- [ ] P9. Cuando arranca una bomba, la ESP32 no se reinicia ni hay caídas de tensión.
- [ ] P10. Prueba de 2 horas continuas con registro de energía.

18.9 Qué debe hacer el firmware

- Leer sensores cada 1 s con promedio, publicar cada 5 s y mantener un perro guardián (watchdog) activo.
- Antes de iniciar el Wi-Fi, dejar todas las salidas digitales en bajo.
- Reconectar el Wi-Fi y MQTT con espera creciente. Publicar `status` con mensaje de última voluntad y `capabilities` retenido al conectar.
- Mantener las reglas locales aunque no haya red: límites de tiempo, bloqueo por nivel y rechazo de comandos con su `reason`.
- Guardar en memoria no volátil la configuración (`unit_id` , broker, límites) y acumular telemetría mientras no hay conexión.
- Si hay internet, sincronizar la hora por NTP; si no, enviar solo `uptime_s`.
- Usuario y contraseña en el broker. No dejarlo abierto a internet.
- Librerías sugeridas: Arduino core para ESP32, ArduinoJson, PubSubClient o AsyncMqttClient, ESPAsyncWebServer y LittleFS (si la ESP32 sirve el panel), WiFiManager (configurar la red desde el móvil) y las librerías del INA219 y del DHT22.
- Actualización por aire (OTA) opcional y protegida con contraseña.

18.10 Seguridad eléctrica y de montaje

- Probar primero en banco con una fuente limitada en corriente, y recién después con la batería.
- Desconectar y descargar antes de tocar el cableado. No manipular conexiones con bombas o válvulas energizadas.
- Conectar siempre las tierras primero y verificar polaridad antes de conectar batería o paneles.
- Hacer cada cambio del circuito analógico por separado y comprobar que el sistema original sigue respondiendo igual antes de continuar.
- Al terminar cada fase, electrónica marca en la sección 15 qué campos del contrato ya son reales y cuáles siguen simulados.